

File organization

- After **Git** and **workflows**, projects need a **layout** newcomers can navigate.

- After **Git** and **workflows**, projects need a **layout** newcomers can navigate.
- Think about **folders**, **data stages**, **naming**, and **README** habits before deeper tooling in later modules.

- A clear structure supports **repeatability**: you and collaborators can revisit, revise, and extend the project.

- A clear structure supports **repeatability**: you and collaborators can revisit, revise, and extend the project.
- Folders separate **concerns**; names stay **chronological** and **descriptive** where it helps.

- A clear structure supports **repeatability**: you and collaborators can revisit, revise, and extend the project.
- Folders separate **concerns**; names stay **chronological** and **descriptive** where it helps.
- Start **simple**; let the tree grow when the project grows.

- A clear structure supports **repeatability**: you and collaborators can revisit, revise, and extend the project.
- Folders separate **concerns**; names stay **chronological** and **descriptive** where it helps.
- Start **simple**; let the tree grow when the project grows.
 - Resist the temptation to over-engineer too early.

Premature optimization is the root of all evil (or at least most of it) in programming. — Knuth

- Consistency across files and projects saves search time and mistakes.

- **Consistency** across files and projects saves search time and mistakes.
- Layout is a form of **communication**: others infer intent from where things live.

- **Consistency** across files and projects saves search time and mistakes.
- Layout is a form of **communication**: others infer intent from where things live.
- Assume work will be **shared** and **reproduced**, even when you work alone: **you** are the future reader.

- Newcomers should find **inputs**, **code**, and **outputs** without a scavenger hunt.

Why organization matters (for others)

- Newcomers should find **inputs**, **code**, and **outputs** without a scavenger hunt.
- Clear structure speeds **collaboration** and **trust** in how the analysis was done.

Why organization matters (for future you)

- Avoid ambiguous names such as `file1.py`, `file-v1.py`, `file-final2.py` competing for attention.

Why organization matters (for future you)

- Avoid ambiguous names such as `file1.py`, `file-v1.py`, `file-final2.py` competing for attention.
- Picking the wrong file by trial and error **breaks reproducibility**.

Why organization matters (for future you)

- Avoid ambiguous names such as `file1.py`, `file-v1.py`, `file-final2.py` competing for attention.
- Picking the wrong file by trial and error **breaks reproducibility**.
- Early conventions pay off when you reopen the repo months later.

- Folder and **directory** mean the same here.

- **Folder** and **directory** mean the same here.
- **File structure**, **hierarchy**, and **schema** are used interchangeably for how directories nest.

- No single layout fits every lab; adapt defaults to your domain.

- No single layout fits every lab; adapt defaults to your domain.
- Balance setup time with time spent actually analyzing.

- No single layout fits every lab; adapt defaults to your domain.
- Balance setup time with time spent actually analyzing.
- Don't worship rules over clarity.

A foolish consistency is the hobgoblin of little minds. — Emerson

1. Do not edit raw data in place; keep an immutable **raw** copy (read-only when possible).

1. **Do not edit raw data** in place; keep an immutable **raw** copy (read-only when possible).
2. Treat analysis like an **assembly line**: raw → cleaned → analyzed → reported; each stage has a clear home.

1. **Do not edit raw data** in place; keep an immutable **raw** copy (read-only when possible).
2. Treat analysis like an **assembly line**: raw → cleaned → analyzed → reported; each stage has a clear home.
3. Separate **scratch** from **finished** code; when a script's job is done, **rename** it usefully and **start a new file** for the next experiment. Don't recycle the same filename for unrelated work.

4. Design for sharing even if collaborators are unlikely: act as if the repo could go public today.

4. Design for sharing even if collaborators are unlikely: act as if the repo could go public today.
5. Internal consistency beats copying someone else's tree blindly.

- FAIR for research outputs and metadata:

- FAIR for research outputs and metadata:
 - **Findable**: clear names, searchable README, stable paths.

Principles checklists (FAIR and related)

- FAIR for research outputs and metadata:
 - **Findable**: clear names, searchable README, stable paths.
 - **Accessible**: documented access method and permissions.

- FAIR for research outputs and metadata:
 - **Findable**: clear names, searchable README, stable paths.
 - **Accessible**: documented access method and permissions.
 - **Interoperable**: standard formats (`csv`, `json`, `parquet`) and clear schemas.

- FAIR for research outputs and metadata:
 - **Findable**: clear names, searchable README, stable paths.
 - **Accessible**: documented access method and permissions.
 - **Interoperable**: standard formats (`csv`, `json`, `parquet`) and clear schemas.
 - **Reusable**: license, provenance, and enough documentation to rerun.

Principles checklists (FAIR and related)

- FAIR for research outputs and metadata:
 - **Findable**: clear names, searchable README, stable paths.
 - **Accessible**: documented access method and permissions.
 - **Interoperable**: standard formats (`csv`, `json`, `parquet`) and clear schemas.
 - **Reusable**: license, provenance, and enough documentation to rerun.
- CARE (especially with human/community data): **Collective benefit**, **Authority to control**, **Responsibility**, **Ethics**.

Principles checklists (FAIR and related)

- FAIR for research outputs and metadata:
 - **Findable**: clear names, searchable README, stable paths.
 - **Accessible**: documented access method and permissions.
 - **Interoperable**: standard formats (`csv`, `json`, `parquet`) and clear schemas.
 - **Reusable**: license, provenance, and enough documentation to rerun.
- CARE (especially with human/community data): **Collective benefit, Authority to control, Responsibility, Ethics**.
- **Reproducibility heuristics** (practical lab version): deterministic scripts, pinned environments, and one-command rebuilds.

- Split functionality so each script or module does **one main job** (clean, plot, fit, etc.).

- Split functionality so each script or module does **one main job** (clean, plot, fit, etc.).
- Folders group **similar roles**: e.g. preprocessing vs model-based analyses.

- Raw data: as acquired; **immutable**, however messy.

- **Raw data**: as acquired; **immutable**, however messy.
- Names from instruments or collaborators can stay in raw; write a **small script** that produces cleaner **intermediate** files so imports can be repeated.

- **Raw data**: as acquired; **immutable**, however messy.
- Names from instruments or collaborators can stay in raw; write a **small script** that produces cleaner **intermediate** files so imports can be repeated.
- **Edited / intermediate**: filters, de-identification, harmonized tables.

- **Raw data:** as acquired; **immutable**, however messy.
- Names from instruments or collaborators can stay in raw; write a **small script** that produces cleaner **intermediate** files so imports can be repeated.
- **Edited / intermediate:** filters, de-identification, harmonized tables.
- **Final analysis inputs:** the dataset your main pipeline actually reads. That separates “latest guess” from trusted inputs and avoids **_final_final** filename chaos.

- Keep **dated notes** or a lab notebook on **what changed** between runs.

- Keep **dated notes** or a lab notebook on **what changed** between runs.
- Helpful experiment table (adapt to your field):

- Keep **dated notes** or a lab notebook on **what changed** between runs.
- Helpful experiment table (adapt to your field):

- Keep **dated notes** or a lab notebook on **what changed** between runs.
- Helpful experiment table (adapt to your field):

Include	Sometimes include
Date	Data source / operator
How the run was done	Whether this is a repeat, re-parameterization, or new method
Notes on outputs	

- Scratch is exploration: plots, checks, ad hoc transforms.

- **Scratch** is exploration: plots, checks, ad hoc transforms.
- Prefer **scripts** over one-off CLI-only steps so results are **repeatable**.

- **Scratch** is exploration: plots, checks, ad hoc transforms.
- Prefer **scripts** over one-off CLI-only steps so results are **repeatable**.
- “**Data to table**”: automate raw → cleaned → tables/figures so tweaks are reruns, not archaeology.

- Source (`src/` or similar): code you rely on for published results; smaller demos can live in `demos/` or a `scratch/` tree if that stays honest.

- Source (`src/` or similar): code you rely on for published results; smaller demos can live in `demos/` or a `scratch/` tree if that stays honest.
- Results (figures, exports, builds): keep separate from source so you can **wipe and rebuild**.

- **Source** (`src/` or similar): code you rely on for published results; smaller demos can live in `demos/` or a `scratch/` tree if that stays honest.
- **Results** (figures, exports, builds): keep separate from source so you can **wipe and rebuild**.
- **Makefiles** or similar help larger projects record **build order** and dependencies.

What every project needs

1. One **root folder** for the project.

What every project needs

1. One **root folder** for the project.
2. A place for **code** (e.g. `src/`).

What every project needs

1. One **root folder** for the project.
2. A place for **code** (e.g. **src/**).
3. A place for **data** (often **data/** with subfolders).

What every project needs

1. One **root folder** for the project.
2. A place for **code** (e.g. **src/**).
3. A place for **data** (often **data/** with subfolders).
4. A **README** (**README.md**) with purpose, setup, and conventions.

What every project needs

1. One **root folder** for the project.
2. A place for **code** (e.g. **src/**).
3. A place for **data** (often **data/** with subfolders).
4. A **README** (**README.md**) with purpose, setup, and conventions.
5. Places for **outputs** and **docs** (e.g. **output/**, **docs/**) once the project earns them.


```
projectName/  
  data/  
    raw/  
    deidentified/  
    processed/  
docs/  
  index.md  
output/  
src/  
  main.py  
  utils.py  
  preprocessing/  
  modelBasedAnalysis/  
README.md
```

1. Create a **dedicated** project folder.

1. Create a **dedicated** project folder.
2. Judge **scale**: quick plot vs multi-year collaboration sets depth of folders.

1. Create a **dedicated** project folder.
2. Judge **scale**: quick plot vs multi-year collaboration sets depth of folders.
3. Let **distinguishing parameters** (time, subject, instrument, condition) show up in paths and names.

1. Create a **dedicated** project folder.
2. Judge **scale**: quick plot vs multi-year collaboration sets depth of folders.
3. Let **distinguishing parameters** (time, subject, instrument, condition) show up in paths and names.
4. Prefer the **simplest** tree you can **inspect and debug**; don't front-load complexity.

- Small / flat: Case study 1 (almost flat)

- Small / flat: Case study 1 (almost flat)
- Exploratory hierarchy: Case study 2 (simple hierarchy)

- Small / flat: Case study 1 (almost flat)
- Exploratory hierarchy: Case study 2 (simple hierarchy)
- Large / multi-paper: Case study 3 (complex hierarchy)

1. Descriptive, especially around versions (`-v2`, not `_final_final`).

1. **Descriptive**, especially around **versions** (`-v2`, not `_final_final`).
2. **Short**; use a **README** for abbreviations.

1. **Descriptive**, especially around **versions** (`-v2`, not `_final_final`).
2. **Short**; use a **README** for abbreviations.
3. Let **parent folders** carry context so filenames stay lean.

- Dates in filenames: YYYY_MM_DD so sorts are chronological (2024_07_01).

Naming (dates and sequences)

- Dates in filenames: YYYY_MM_DD so sorts are chronological (2024_07_01).
- Zero-pad numeric sequences: 01 .. 10, not 1 .. 10.

Naming (dates and sequences)

- Dates in filenames: YYYY_MM_DD so sorts are chronological (2024_07_01).
- Zero-pad numeric sequences: 01 .. 10, not 1 .. 10.
- Match **team conventions** when you join a lab.

- The full path (folders + file) tells humans **which idea** each segment carries.

- The full path (folders + file) tells humans **which idea** each segment carries.
- Folders usually name **one** idea; filenames may combine several.

Multiple conventions can make sense:

Multiple conventions can make sense:

- `participant01_rdmStudy-v2.1_2024_07_01.csv`

Multiple conventions can make sense:

- `participant01_rdmStudy-v2.1_2024_07_01.csv`
 - Ideas: participant ID · experiment (+ version) · acquisition date.

Multiple conventions can make sense:

- `participant01_rdmStudy-v2.1_2024_07_01.csv`
 - Ideas: participant ID · experiment (+ version) · acquisition date.
- `rdmStudy/v2.1/participant01_2024_07_01.csv`

Multiple conventions can make sense:

- `participant01_rdmStudy-v2.1_2024_07_01.csv`
 - Ideas: participant ID · experiment (+ version) · acquisition date.
- `rdmStudy/v2.1/participant01_2024_07_01.csv`
 - Ideas: experiment (+ version) · participant ID · acquisition date.

Multiple conventions can make sense:

- `participant01_rdmStudy-v2.1_2024_07_01.csv`
 - Ideas: participant ID · experiment (+ version) · acquisition date.
- `rdmStudy/v2.1/participant01_2024_07_01.csv`
 - Ideas: experiment (+ version) · participant ID · acquisition date.
- `rdmStudy/v2.1/participant01/2024_07_01.csv`

Multiple conventions can make sense:

- `participant01_rdmStudy-v2.1_2024_07_01.csv`
 - Ideas: participant ID · experiment (+ version) · acquisition date.
- `rdmStudy/v2.1/participant01_2024_07_01.csv`
 - Ideas: experiment (+ version) · participant ID · acquisition date.
- `rdmStudy/v2.1/participant01/2024_07_01.csv`
 - Ideas: experiment (+ version) · participant ID · acquisition date.

Multiple conventions can make sense:

- `participant01_rdmStudy-v2.1_2024_07_01.csv`
 - Ideas: participant ID · experiment (+ version) · acquisition date.
- `rdmStudy/v2.1/participant01_2024_07_01.csv`
 - Ideas: experiment (+ version) · participant ID · acquisition date.
- `rdmStudy/v2.1/participant01/2024_07_01.csv`
 - Ideas: experiment (+ version) · participant ID · acquisition date.
- Mostly you want to manage the number of files in a folder with a simple hierarchy.

- Put the **most important** discriminator **first** (often experiment vs date depends on design).

- Put the **most important** discriminator **first** (often experiment vs date depends on design).
- **Version** tokens (`_v1`, `_v2`) come **last** so they don't obscure the main identity.

- Put the **most important** discriminator **first** (often experiment vs date depends on design).
- **Version** tokens (`_v1`, `_v2`) come **last** so they don't obscure the main identity.
- Chaotic version suffixes undermine confidence in the work.

- Separate word groups so ideas don't blur: **`process_data.py`**, **`processedData.csv`**.

Delimiters: snake_case and camelCase

- Separate word groups so ideas don't blur: **process_data.py**, **processedData.csv**.
- camelCase: capitalize subsequent words only (**firstName**, **rdmData**).

Delimiters: snake_case and camelCase

- Separate word groups so ideas don't blur: **process_data.py**, **processedData.csv**.
- **camelCase**: capitalize subsequent words only (**firstName**, **rdmData**).
- Mixing styles **within** one name can work if the pattern stays **readable** (examples from the MIT notes: **20190701.GeorgeSun.v1.csv** vs **20190701_davidLarson_v1.csv**).

No spaces in project paths

- Spaces in repo paths break **shell** and **tooling** workflows; use -, _, or case switches instead.

- MIT CommKit: File structure

- MIT CommKit: File structure
- Russell A. Poldrack, [Better Code, Better Science](#): Project organization